CS 630: Software Testing and Quality Assurance

1-2 Paper: Comparing Human-Centric and Automated Testing Approaches

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

July 9, 2026

Human-Centric and Tool-Assisted Testing in Software Development

Software testing and quality assurance are essential parts of the software development life cycle because they help ensure that software is reliable, functional, secure, and useful for end users. Human-centric testing and tool-assisted testing both contribute to software quality, but they are most effective when used for the right purpose. Human-centric testing relies on human judgment, creativity, and observation, while tool-assisted testing uses automation tools, scripts, and frameworks to run tests faster and more consistently.

Human-centric testing includes manual testing, exploratory testing, usability testing, user acceptance testing, and stakeholder reviews. Its key strength is that human testers can think critically, notice unexpected behavior, evaluate user experience, and adapt while testing. Exploratory testing is especially valuable because testers learn about the software while actively designing and executing tests. Research notes that exploratory testing remains widely used in agile teams because it fills gaps left by automated testing and relies on tester creativity and expertise (Copche et al., 2024). Human-centric testing is also important when evaluating usability, accessibility, emotional response, and whether software meets real user needs.

Tool-assisted testing, often called automated testing, uses software tools to execute predefined test cases, compare actual results with expected results, and report failures. Automated tests are commonly used for unit testing, regression testing, integration testing, performance testing, smoke testing, and continuous integration/continuous delivery pipelines. Automation allows teams to test faster, more frequently, and at a larger scale than manual testing alone (ISTQB Global Exams, n.d.). Automated testing also supports continuous delivery by allowing code changes to be checked automatically before release (Atlassian, n.d.).

Tool-assisted testing has several advantages. It improves speed because automated tests can run much faster than manual tests. It also improves consistency because the same test can be repeated the same way each time. This is especially helpful for regression testing, where teams need to confirm that new code has not broken existing functionality. Automation can also expand test coverage across browsers, operating systems, devices, and large data sets. However, tool-assisted testing also has limitations. Automated tests require planning, scripting, maintenance, and technical skills. If requirements or user interfaces change often, automated tests may break and require updates. Poor automation can also create false confidence if the tests only check expected paths and miss unusual user behavior or usability problems (ASTQB, n.d.).

Human-centric testing also has important advantages. Human testers can use judgment, intuition, and real-world thinking to find issues that tools may be missed. They can notice confusing workflows, unclear instructions, visual design problems, and unexpected user behavior. Human-centric testing is especially useful for new features, early prototypes, and software designed for a broad audience. However, manual testing can be slower, more expensive over time, and less consistent because results may vary depending on the tester's experience, focus, and interpretation. It is also difficult for humans to repeat large regression test suites quickly after every code changes.

When comparing both approaches, effectiveness depends on the quality assurance goal. For test coverage, tool-assisted testing is stronger because it can run many test cases repeatedly across different environments. For error discovery, both approaches are useful: automation is effective for known defects and regression issues, while human-centric testing is better for discovering unexpected problems. For adaptability to change, human-centric testing is often stronger because people can adjust their testing approach as the product changes. Automated testing is less

adaptable unless test scripts are updated. For usability, human-centric testing is more effective because usability depends on human perception, ease of use, accessibility, and satisfaction.

A scenario where human-centric testing would be more effective is the early testing of a new mobile app for the public. If the team is still refining navigation, screen layout, and user flow, human testers and real users would provide better feedback than automation. For example, automation could confirm that a button works, but a human tester could explain that the button is hard to find, the instructions are confusing, or the workflow feels frustrating.

A blended approach would be most effective for a web-based healthcare scheduling system used by medical office staff and patients. The software platform would include a web application, database, and mobile-responsive interface. The team might include programmers, QA testers, database analysts, business analysts, and healthcare subject matter experts. Programmer-focused testing could include automated unit tests, API tests, database validation, and regression tests. Customer-facing testing could include manual usability testing, user acceptance testing, and exploratory testing with office staff and patients. Automated testing would help ensure that login, appointment scheduling, database updates, and notifications continue working after each code change. Human-centric testing would help confirm that patients and staff can understand and use the system correctly.

In general, neither human-centric nor tool-assisted testing is enough by itself. Automated testing provides speed, consistency, and broad regression coverage, while human-centric testing provides creativity, adaptability, and insight into user experience. The strongest quality assurance strategy blends both approaches so that software is not only technically correct but also useful, understandable, and reliable for the people who depend on it.

References

ASTQB. (n.d.). *ASTQB Test Automation Pro official ISTQB exam*. https://astqb.org/astqb-test-automation-pro/

Atlassian. (n.d.). *Software testing in continuous delivery*. https://www.atlassian.com/continuous-delivery/software-testing

Copche, R., Durelli, V. H. S., & Durelli, R. S. (2024). *Can a chatbot support exploratory software testing?* SCITEPRESS. https://www.scitepress.org/Papers/2024/125724/125724.pdf

ISTQB Global Exams. (n.d.). *What is test automation?* https://istqbglobalexams.com/what-is-test-automation/